

07_NETWORK E SOCKET

1. RICHIAMI DI RETE

Computer communication: scambio di informazioni fra diversi computer a scopo di azioni cooperative

Computer network: insieme di due o più computer interconnessi attraverso un communication network

Nel parlare di computer communication e computer network, escono fuori due concetti essenziali: protocolli e architetture di computer communication (o architettura di protocollo)

Protocollo: insieme di regole che governano lo scambio di dati fra due entità. Vengono usati per la comunicazione fra entità in diversi sistemi.

Entità: qualsiasi cosa capace di inviare o ricevere informazioni.

Sistema: oggetto fisico che contiene una o più entità.

Per comunicare due entità devono “parlare la stessa lingua”.

Elementi chiave dei procolli sono:

- Sintassi, include data format e signal levels
- Semantica, include informazioni di controllo per la coordinazione e error handling
- Timing, include speed matching e sequenziamento

Protocol Architecture: insieme strutturato di moduli che implementa le funzioni di comunicazione

Ci deve essere un alto grado di cooperazione fra i due computer system. Invece che implementare tutto come un singolo modulo, le task sono divise in subtask.

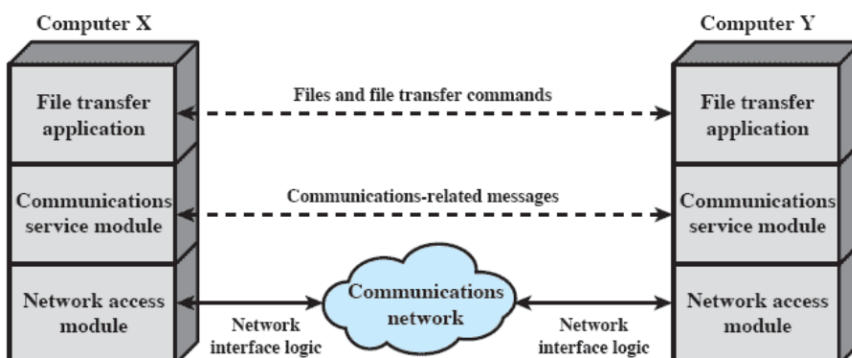
Ad esempio:

Il **file transfer module** contiene tutta la logica che è unica per l'applicazione file transfer.

Il **communications service module** deve assicurare che i due computer system siano attivi e pronti al trasferimento di dati, e pronti a tenere traccia dei dati che si stanno scambiando, per assicurare la consegna.

La logica per interagire realmente col network è messa dentro un separato **network access module**.

File-transfer architecture:



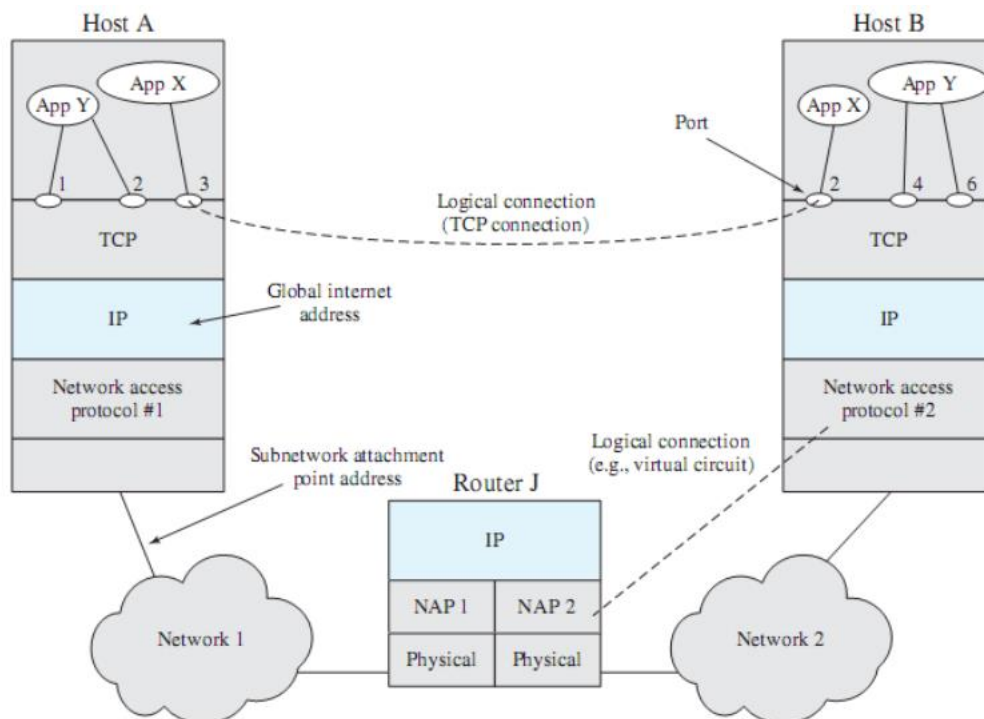
TCP/IP protocol architecture: risultato di ricerca e sviluppo dei protocolli condotta sul packet-switched network sperimentale ARPANET. Chiamato TCP/IP protocol suite.

Le communication task per TCP/IP possono essere organizzate in 5 livelli indipendenti:

- **application layer**
- **transport layer**, host-to-host. Livello comune condiviso fra tutte le applicazioni, contiene il meccanismo per garantire affidabilità nello scambio di dati.
Il **Transmission Control Protocol (TCP)** è il protocollo più diffuso per provvedere questa funzione.
- **internet layer**, la sua funzione è di permettere ai dati di spostarsi attraverso molteplici network interconnessi. A questo livello si usa l'**Internet Protocol (IP)** per fornire funzioni di routing attraverso molteplici network.
IP è implementato non solo negli end system, ma anche nei router.
Router: processore che connette due network. La sua funzione primaria è di passare dati da un network ad un altro.
- **network access layer**, si occupa dello scambio di dati fra un end system e il network al quale è attaccato. Il software specifico usato a questo livello dipende dal tipo di network che si userà
- **physical layer**, copre l'interfaccia fisica fra un data transmission device e un mezzo di trasmissione o network. Questo livello si occupa di:
 - specificare le caratteristiche del mezzo di trasmissione
 - la natura dei segnali
 - il data rate

Application layer:

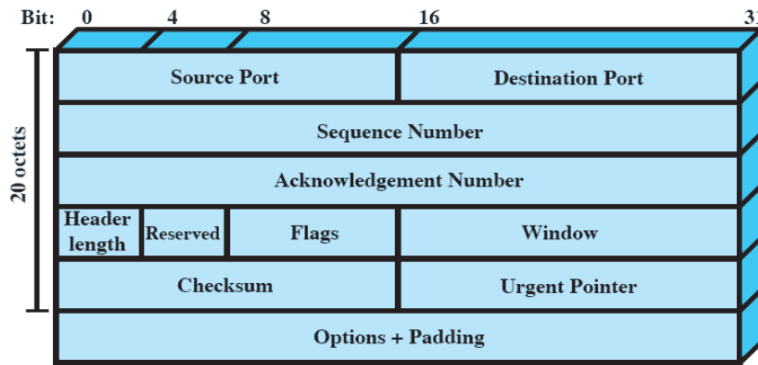
contains the logic needed to support the various user applications. For each different type of application, a separate module is needed that is peculiar to that application.



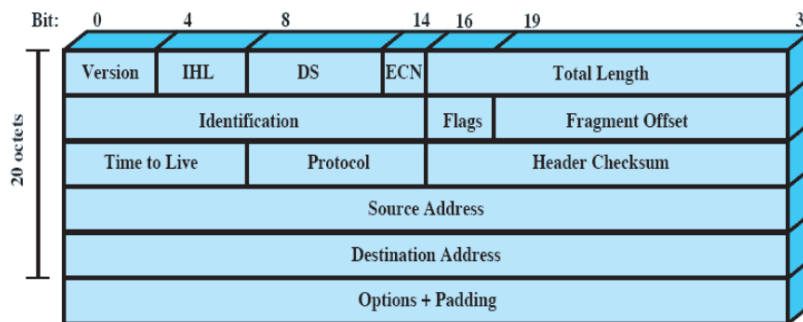
Every entity in the overall system must have a unique address. Two levels of addressing are needed:

- Each host on a network must have a unique global internet address (this address is used by IP for routing and delivery).
- Each application within a host must have an address that is unique within the host, this allows the host-to-host protocol (TCP) to deliver data to the proper process. These addresses are known as **ports**.

TCP header:



IP header (IPv4):



TCP/IP applications: a number of applications have been standardized to operate on top of TCP.

Examples:

- Simple Mail Transfer Protocol (SMTP), provides a basic electronic mail facility, features include mailing lists, return receipts, and forwarding
- File Transfer Protocol (FTP), used to send files from one system to another under user command, both text and binary files are accommodated, provides features for controlling user access
- Secure Shell (SSH), provides a secure remote logon capability, which enables a user at a terminal or personal computer to log on to a remote computer and function as if directly connected to that computer. Supports file transfer between the local host and a remote server. SSH traffic is carried on a TCP connection

UDP: minimum protocol mechanism. It is connectionless, there's no guarantees about delivery, preservation of sequence, nor protection against duplication.

Useful for example for transaction-oriented applications.

Multicast support.

UDP header:



2. SOCKET

Socket: Developed as the **Berkeley Sockets Interface (BSI)**. Enables communication between a client and server process.

May be either connection oriented or connectionless.

Can be considered an endpoint in communication.

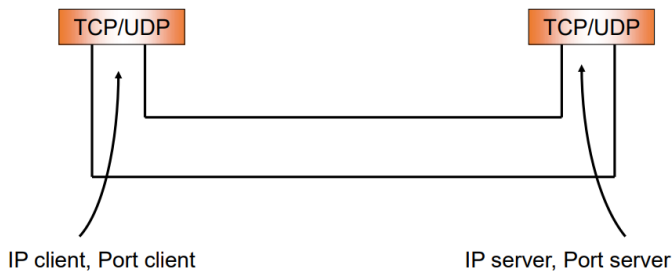
The BSI transport layer interface is the de-facto standard API for developing networking applications.

Windows sockets (WinSock) are based on the Berkeley specification.

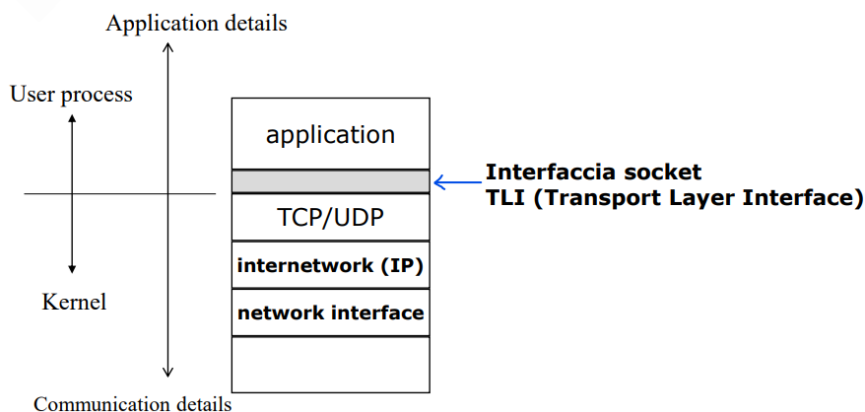
IP addresses identify the respective host systems.

The concatenation of a port value and an IP address forms a socket, which is unique throughout the Internet. Socket: IP address + port

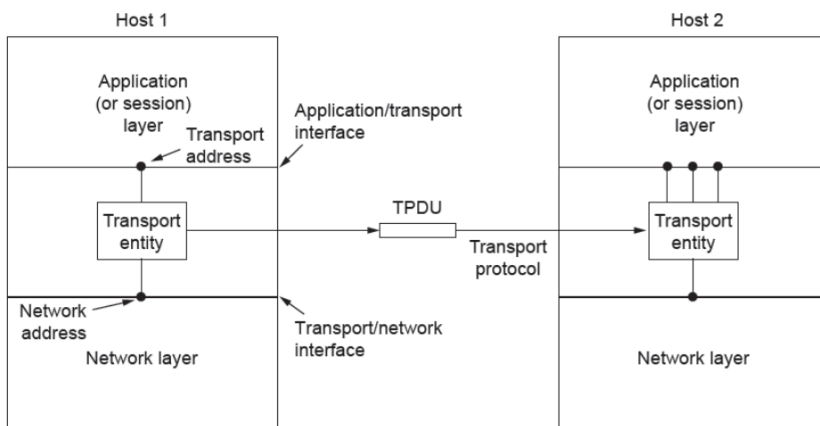
Quadrupla (IP client, Port client, IP server, Port server):



L'interfaccia socket (**TLI, transport layer interface**) rappresenta l'interfaccia a livello di trasporto



Servizi forniti al livello superiore (application layer):



Ci sono diversi tipi di socket:

- **Stream socket:** makes use of TCP, provides a connection-oriented reliable data transfer
- **Datagram socket:** make use of UDP, delivery is not guaranteed, nor is order necessarily preserved
- **Raw socket,** allow direct access to lower-layer protocols

Caratteristiche di base delle socket:

- A socket represents an end-point for an IP network connection, what the application layer “plugs into” (programmer cares about API)
- An end point (socket) is determined by two things: Host address (IP address is Network Layer) and Port number (is Transport Layer)
- Two end-points determine a connection: socket pair

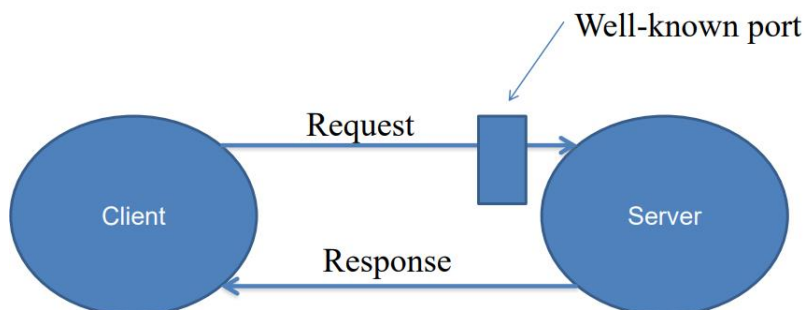
Ad esempio:

206.62.226.35,p21 + 198.69.10.2,p1500

206.62.226.35,p21 + 198.69.10.2,p1499

Approccio client-server:

Il client invia una richiesta al server su una porta conosciuta (o io conosco la porta, oppure il server mi espone una porta per i servizi noti, a cui posso chiedere qual è la porta che voglio).

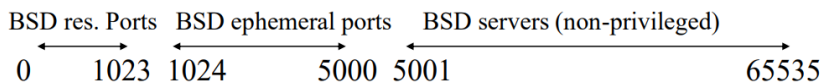


Le porte generalmente sono numeri che vanno da 0 a 65535.

In Solaris, Linux, le prime 1023 sono riservate, quelle da 1024 a 5000 sono effimere (devono avere vita breve). Quelle che possiamo usare noi sono da 5001 a 65535.

- 0-1023 “reserved”, must be root
- 1024 - 5000 “ephemeral” (short-lived ports assigned automatically by the OS to clients)
- however, many systems allow > 3977 ephemeral ports due to the number of increasing handled by a single PC. (Sun Solaris provides 30,000 in the last portion of BSD non-privileged)

Well-known, reserved services /etc/services: [ftp 21/tcp](#), telnet 23/tcp, finger 79/tcp, snmp 161/udp

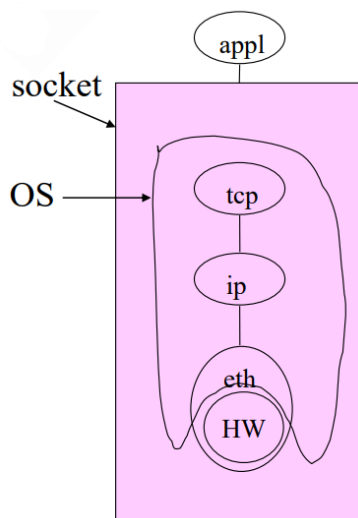


Several client program needs to be a server at the same time (rlogin, rsh) as a part of the client-server authentication. These clients call the library function `rresvport` to create a socket and to assign an unused port in the range 512-1024.

Socket e OS:

User sees “descriptor”, integer index

- like: `FILE *`, or file index from `open()`
- returned by `socket()` call (more later)



Primitive per un transport service (su socket):

- **Listen**, non invia nessun pacchetto, si blocca finché qualche altro processo prova a connettersi
- **Connect**, invia pacchetto “Connection Req.”, cerca attivamente di stabilire una connessione
- **Send**, invia pacchetto “Data”, invia informazioni
- **Receive**, non invia nessun pacchetto, si blocca finché non riceve un pacchetto di dati (Data)
- **Disconnect**, invia un pacchetto di “Disconnect Req.”, l’host corrente vuole rilasciare la connessione.

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

Il server fa una listen per aspettare eventuali client.

I client fanno una connect per cercare di connettersi al server. Con la connect un client trasmette le proprie informazioni (come indirizzo e porta).

Struttura della socket:

```

struct in_addr {
    in_addr_t s_addr; /* 32-bit IPv4 addresses */
};
/* intero a 32 bit per IPv4 */

struct sockaddr_in {
    uint8_t sin_len; /* length of structure */
    sa_family_t sin_family; /* AF_INET */
    in_port_t sin_port; /* TCP/UDP Port num */
    struct in_addr sin_addr; /* IPv4 address (above) */
    char sin_zero[8]; /* unused */
};
/* lunghezza socket */
/* protocollo */
/* porta */
/* indirizzo IP */
/* inutilizzato */

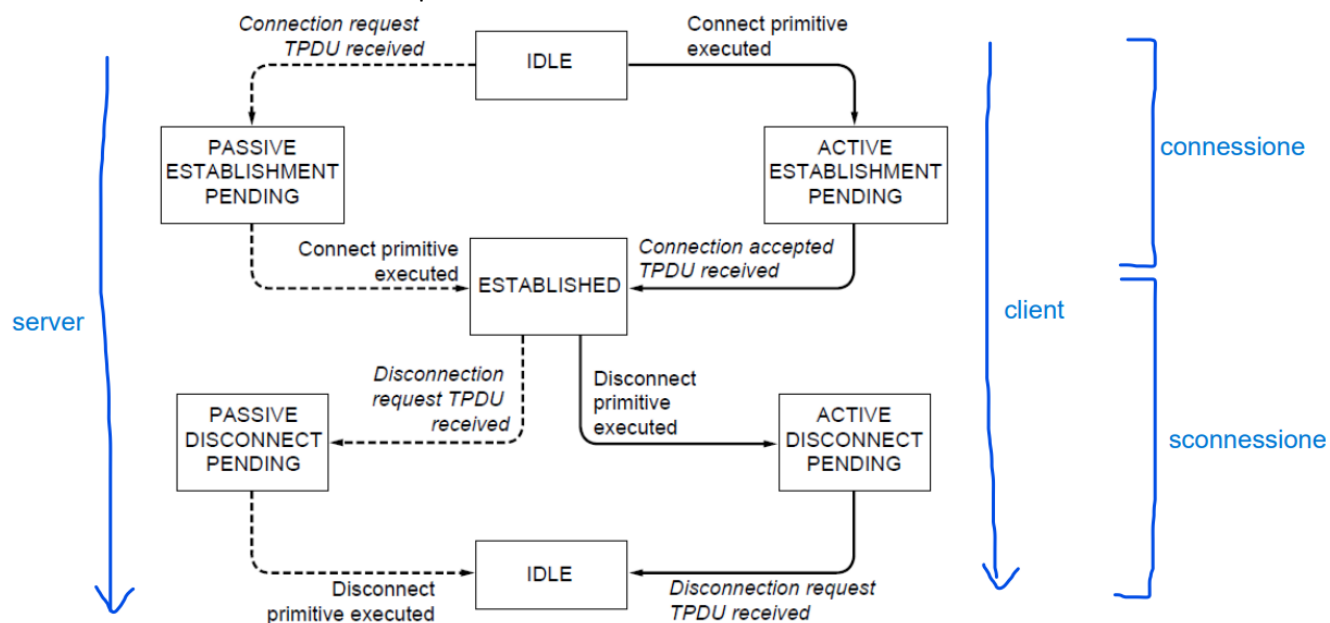
```

Si fa una inizializzazione a 0 (e.g., bzero, memset) before using sockaddr.

A state diagram for a simple connection management scheme:

transitions labeled in *italic* are caused by packet arrivals. The solid lines show the client's state sequence.

The dashed lines show the server's state sequence.



Primitive delle socket per TCP (**Berkeley Sockets**):

- **Socket**, crea un nuovo communication end point (nuova socket)
- **Bind**, associa un indirizzo locale ad una socket
- **Listen**, annuncia che è pronto ad accettare connessioni, viene data una dimensione della coda
- **Accept**, stabilisce passivamente una connessione in arrivo (accetta una richiesta di connessione)
- **Connect**, cerca di stabilire attivamente una connessione (invia una richiesta di connessione)
- **Send**, invia dati sulla connessione
- **Receive**, riceve dati dalla connessione
- **Close**, rilascia la connessione

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Con accept accetto la connessione, avviene quindi la connessione vera e propria.

For a stream socket (TCP), once the socket is created, a connection must be set up to a remote socket. One side functions as a client and requests a connection to the other side, which acts as a server.

Client:

issues a **connect()** that specifies both a local socket and the address of a remote socket
→ once a connection is set up, **getpeername()** can be used to find out who is on the other end of the connected stream socket

Server:

a server application issues a **listen()**, indicating that the given socket is ready to accept incoming connections
→ each incoming connection is placed in this queue until a matching **accept()** is issued by the server side

We need a structure to hold address information.

Functions pass address from user to OS:

`bind()`

`connect()` (*TCP only*)

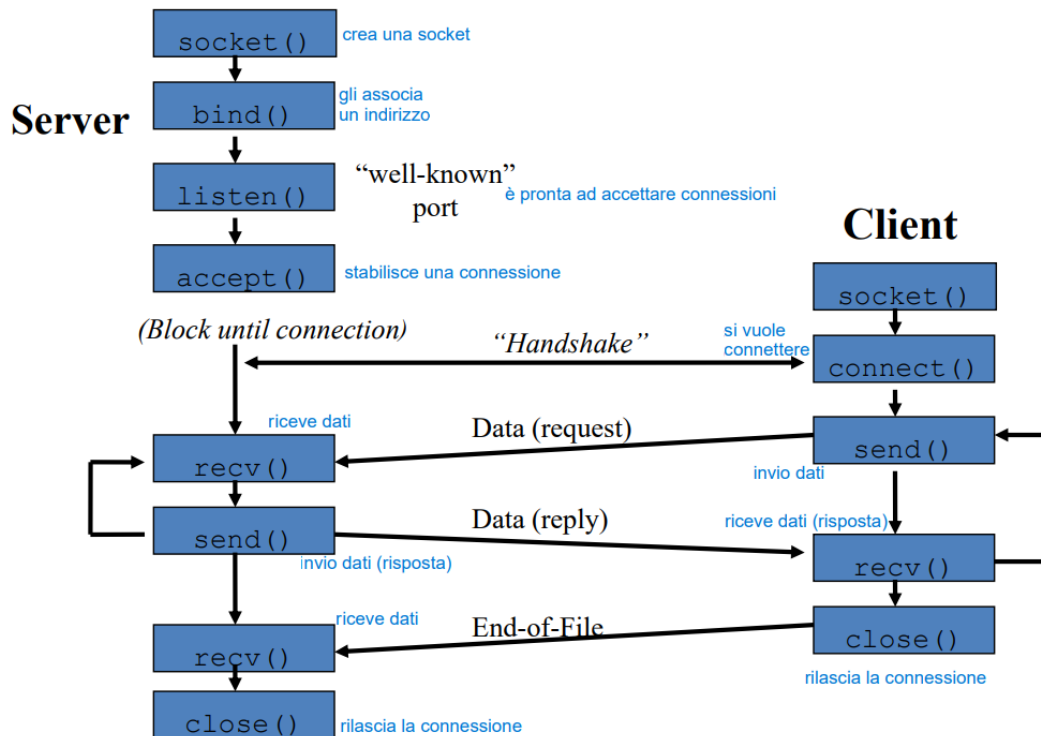
`sendto()` (*UDP only*)

Functions pass address from OS to user:

`accept()` (*TCP only*)

`recvfrom()` (*UDP only*)

TCP client-server:



Socket setup: the first step in using Sockets is to create a new socket using the `socket()` API. Returns an integer that identifies the socket (similar to a UNIX file descriptor).

Includes three parameters:

- protocol family is always `AF_INET` for the TCP/IP protocol suite
- type specifies whether this is a stream or datagram socket
- protocol specifies either TCP or UDP

`int socket(int family, int type, int protocol)`: create a socket, giving access to transport layer service.

- family is one of:
 - > `AF_INET` (IPv4), `AF_INET6` (IPv6), `AF_LOCAL` (local Unix)
 - > `AF_ROUTE` (access to routing tables), `AF_KEY` (new, for encryption)
- type is one of:
 - > `SOCK_STREAM` (TCP), `SOCK_DGRAM` (UDP)
 - > `SOCK_RAW` (for special IP packets, PING, etc. Must be root)
- protocol is 0 (used for some raw socket options)

Upon success returns socket descriptor (Integer, like file descriptor). Return -1 if failure.

`int bind(int sockfd, const struct sockaddr *myaddr, socklen_t addrlen)`: assign a local protocol address ("name") to a socket.

- `sockfd` is socket descriptor from `socket()`
- `myaddr` is a pointer to address struct with port number and IP address (if port is 0, then host OS will pick ephemeral port)
- `addrlen` is length of structure

returns 0 if ok, -1 on error (`EADDRINUSE`, "Address already in use")

Nelle chiamate che passano la struttura indirizzo da processo utente a OS (esempio bind) viene passato un puntatore alla struttura ed un intero che rappresenta il sizeof della struttura (oltre ovviamente ad altri parametri). In questo modo l'OS kernel sa esattamente quanti byte deve copiare nella sua memoria.

Nelle chiamate che passano la struttura indirizzo dall'OS kernel al processo utente (esempio accept) vengono passati nella system call eseguita dall'utente un puntatore alla struttura ed un puntatore ad un intero in cui è stata preinserita la dimensione della struttura indirizzo. In questo caso, sulla chiamata della system call, il kernel dell'OS sa la dimensione della struttura quando la va a riempire e non ne oltrepassa i limiti. Quando la system call ritorna inserisce nell'intero la dimensione di quanti byte ha scritto nella struttura.

Questo modo di procedere è utile in system call come la select e la getsockopt che vedremo durante le esercitazioni.

int listen(int sockfd, int backlog): change socket state for TCP server.

- sockfd is socket descriptor from socket()
- backlog is maximum number of incomplete connections (historically 5, implementation might use it just as a hint)

int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen): Return next completed connection.

- sockfd is socket descriptor from socket()
- cliaddr and addrlen return protocol address from client

If used with fork(), can create concurrent server.

If used with pthread_create() can create a multithread server.

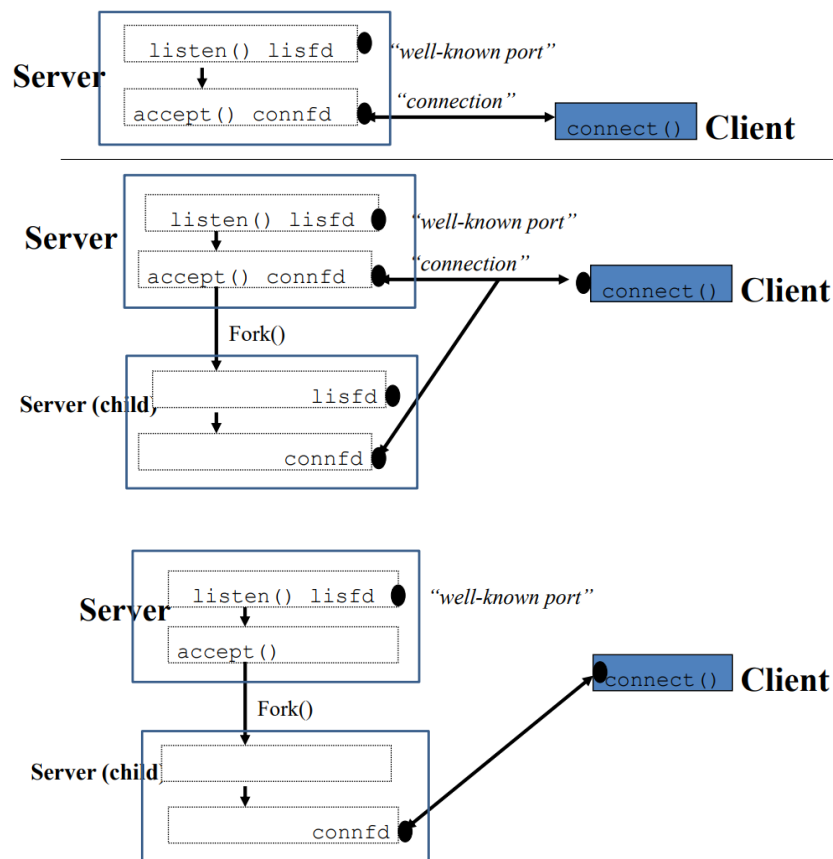
Returns new descriptor, created by OS.

On error, -1 is returned, and errno is set appropriately.

accept() + fork():

```
lisfd = socket(...);    crea socket
bind(lisfd, ...);       associa al processo locale la socket
listen(lisfd, 5);       sono pronto ad accettare connessioni
while(1) {
    connfd = accept(lisfd, ...);  accetta una richiesta di
                                   connessione sulla socket
                                   locale
    if (pid = fork() == 0) {      processo figlio (server concorrente)
        close(lisfd);            rilascia la connessione con la socket locale lisfd
        doit(connfd);            faccio quello che devo fare sui dati ricevuti
                                   (send, receive ed elaborazione dei dati)
        close(connfd);           rilascia i dati ricevuti
        _exit(0);
    }
    close(connfd);
}
```

Nel ciclo: il processo padre accetta una connessione in entrata, crea un processo figlio, che si occupa del servizio di server. Lui poi chiude la connessione appena accettata (di cui si sta occupando il figlio), e crea un nuovo figlio.



int close(int sockfd): close socket for use.

- sockfd is socket descriptor from socket()
- closes socket for reading/writing

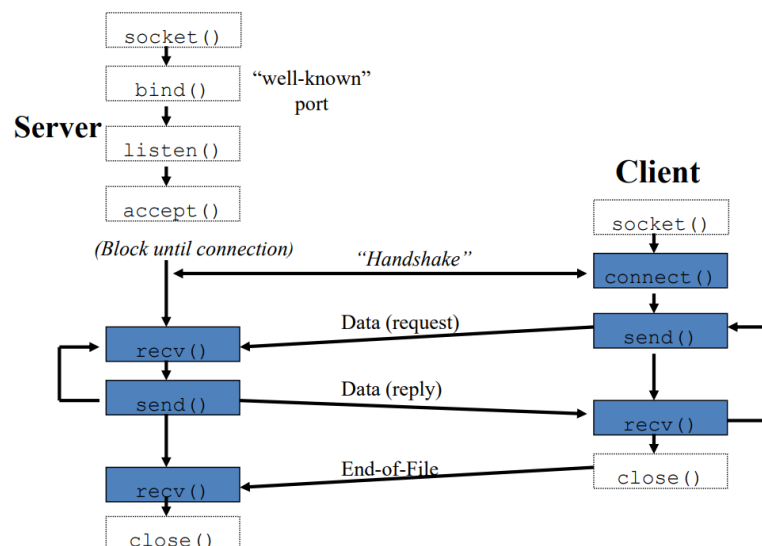
Returns (doesn't block).

Attempts to send any unsent data.

Socket option `SO_LINGER` timeout: block until data sent or discard any remaining data.

Returns -1 if error.

TCP client-server:



int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen): connect to server.

- sockfd is socket descriptor from socket()
- servaddr is a pointer to a structure with port number and IP address (must be specified, unlike bind())
- addrlen is length of structure
- client doesn't need bind(), OS will pick ephemeral port

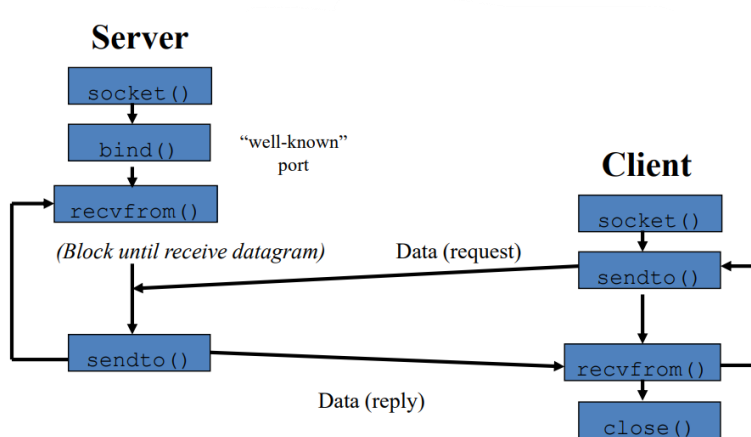
returns 0 if ok, -1 on error.

int recv(int sockfd, void *buff, size_t numBytes, int flags) ,

int send(int sockfd, void *buff, size_t numBytes, int flags): same as read() and write() but for flags.

- MSG_DONTWAIT (this send non-blocking)
- MSG_OOB (out of band data, 1 byte sent ahead)
- MSG_PEEK (look, but don't remove)
- MSG_WAITALL (don't give me less than max)
- MSG_DONTROUTE (bypass routing table)

UDP client-server:



No "handshake", no simultaneous close, no fork() for concurrent servers.

int recvfrom(int sockfd, void *buff, size_t numBytes, int flags, struct sockaddr *from, socklen_t *addrlen) ,

int sendto(int sockfd, void *buff, size_t numBytes, int flags, const struct sockaddr *to, socklen_t addrlen):

sending and receiving, same as `recv()` and `send()` but for addr

`recvfrom` fills in address of where packet came from.

`sendto` requires address of where sending packet to.

int gethostname(char *hostname, int bufferLength): get the name of the host the program is running on.

Upon return, `hostname` holds the name of the host. `bufferLength` provides a limit on the number of bytes that `gethostname()` can write to `hostname`.

Vediamo due Internet Address Library Routines:

unsigned long inet_addr(char *address): converts address in dotted form to a 32-bit numeric value in network byte order (for example "128.173.41.41").

Genera l'indirizzo IP corrispondente all'host name.

char* inet_ntoa(struct in_addr address): struct in_addr.

address.s_addr is the long int representation.

struct hostent* getbyhostname(const char *hostname): given a host name (such as acavax.lynchburg.edu) get host information.

- char* h_name; // official name of host
- char** h_aliases; // alias list
- short h_addrtype; // address family (e.g., AF_INET)
- short h_length; // length of address (4 for AF_INET)
- char** h_addr_list; // list of addresses (null pointer terminated)

struct hostent* gethostbyaddr(const void *addr, int len, int type): get the name of the host given its address.

- *addr is a pointer to a struct of a type depending on the address type (es: struct in_addr)
- len is 4 if type is AF_INET
- type is the address family (e.g., AF_INET)

Socket di Windows: **WinSock**

Derived from Berkeley Sockets (Unix), includes many enhancements for programming in the windows environment.

Open interface for network programming under Microsoft Windows (API freely available, multiple vendors supply WinSock, source and binary compatibility).

Collection of function calls that provide network services.

Differenze fra Berkeley Sockets e WinSock:

Berkeley	WinSock
bzero()	memset()
close()	closesocket()
read()	not required
write()	not required
ioctl()	ioctlsocket()

WinSock1.1 supports three different modes:

- Blocking mode, socket functions don't return until their jobs are done, same as Berkeley sockets
- Non-blocking mode, calls such as accept() don't block, but simply return a status
- Asynchronous mode, uses Windows messages (FD_ACCEPT: connection pending
FD_CONNECT: connection established, ...).

WinSock2: Supports protocol suites other than TCP/IP (DecNet, IPX/SPX, OSI). Supports network-protocol independent applications.

It is backward compatible with WinSock 1.1

WinSock2 uses different files (winsock2.h, different DLL (WS2_-32.DLL)).

API changes:

- accept() becomes WSAAccept()
- connect() becomes WSAConnect()
- inet_addr() becomes WSAAddressToString()
- ...

Obviously a well done server must include:

- Process/thread generation
- Continuous listening
- Repeated communication

Stream sockets (TCP) in Java:

Server:

```
public class GreetServer {
    private ServerSocket serverSocket;
    private Socket clientSocket;
    private PrintWriter out;
    private BufferedReader in;

    public void start(int port) {
        serverSocket = new ServerSocket(port);
        clientSocket = serverSocket.accept();
        out = new PrintWriter(clientSocket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
        String greeting = in.readLine();
        if ("hello server".equals(greeting)) {
            out.println("hello client");
        }
        else {
            out.println("unrecognised greeting");
        }
    }

    public void stop() {
        in.close();
        out.close();
        clientSocket.close();
        serverSocket.close();
    }

    public static void main(String[] args) {
        GreetServer server=new GreetServer();
        server.start(6666);
        server.stop();
    }
}
```

Client:

```
public class GreetClient {
    private Socket clientSocket;
    private PrintWriter out;
    private BufferedReader in;

    public void startConnection(String ip, int port) {
        clientSocket = new Socket(ip, port);
        out = new PrintWriter(clientSocket.getOutputStream(), true);
        in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
    }

    public String sendMessage(String msg) {
        out.println(msg);
        String resp = in.readLine();
        return resp;
    }

    public void stopConnection() {
        in.close();
        out.close();
        clientSocket.close();
    }

    public static void main(String[] args) {
        GreetClient client = new GreetClient();
        client.startConnection("127.0.0.1", 6666);
        String response = client.sendMessage("hello server");
        assertEquals("hello client", response);
        client.stopConnection()
    }
}
```

UDP sockets in Java:

Server:

```
class UDPServer {
    public static void main(String args[]) throws Exception
    {

        DatagramSocket serverSocket = new DatagramSocket(9876);
        byte[] receiveData = new byte[1024];
        byte[] sendData = new byte[1024];

        while(true){

            DatagramPacket receivePacket =
                new DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);

            String sentence = new String(receivePacket.getData());
            InetAddress IPAddress = receivePacket.getAddress();
            int port = receivePacket.getPort();

            String capitalizedSentence = sentence.toUpperCase();
            sendData = capitalizedSentence.getBytes();

            DatagramPacket sendPacket =
                new DatagramPacket(sendData, sendData.length, IPAddress, port);

            serverSocket.send(sendPacket);
        }
    }
}
```

Client:

```
class UDPClient {
    public static void main(String args[]) throws Exception
    {

        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));

        DatagramSocket clientSocket = new DatagramSocket();

        InetAddress IPAddress = InetAddress.getByName("hostname");

        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];

        String sentence = inFromUser.readLine();
        sendData = sentence.getBytes();
        DatagramPacket sendPacket =
            new DatagramPacket(sendData, sendData.length, IPAddress, 9876);

        clientSocket.send(sendPacket);

        DatagramPacket receivePacket =
            new DatagramPacket(receiveData, receiveData.length);

        clientSocket.receive(receivePacket);

        String modifiedSentence = new String(receivePacket.getData());

        System.out.println("FROM SERVER:" + modifiedSentence);
        clientSocket.close();
    }
}
```

Stream sockets (TCP) in Python:

Server:

```
#!/usr/bin/env python3
# server

import socket

HOST = '127.0.0.1' # Standard loopback interface address (localhost)
PORT = 65432      # Port to listen on (non-privileged ports are > 1023)

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind((HOST, PORT))
s.listen()
conn, addr = s.accept()
with conn:
    print('Connected by', addr)
    while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data)
s.close()
```


Client:

```
#!/usr/bin/env python3
# client

import socket

HOST = '127.0.0.1' # The server's hostname or IP address
PORT = 65432      # The port used by the server

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((HOST, PORT))
s.sendall(b'Hello, world')
data = s.recv(1024)

print('Received', repr(data))
s.close()
```

UDP sockets in Python:

Server:

```
#!/usr/bin/env python3
# server

import socket

UDP_IP = "127.0.0.1"
UDP_PORT = 5005

sock = socket.socket(socket.AF_INET, # Internet
                     socket.SOCK_DGRAM) # UDP
sock.bind((UDP_IP, UDP_PORT))

while True:
    data, addr = sock.recvfrom(1024) # buffer size is 1024 bytes
    print "received message:", data
    sock.sendto(data, addr)
```

Client:

```
#!/usr/bin/env python3
# client

import socket

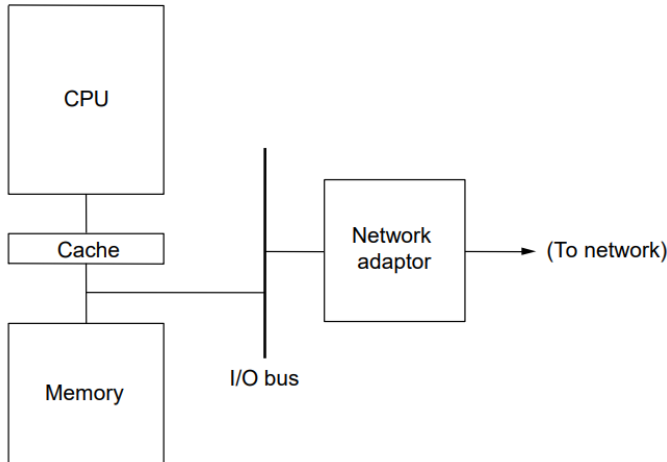
UDP_IP = "127.0.0.1"
UDP_PORT = 5005
MESSAGE = "Hello, World!"

print "UDP target IP:", UDP_IP
print "UDP target port:", UDP_PORT
print "message:", MESSAGE

sock = socket.socket(socket.AF_INET, # Internet
                     socket.SOCK_DGRAM) # UDP
sock.sendto(MESSAGE, (UDP_IP, UDP_PORT))
data, addr = sock.recvfrom(1024)
sock.close()
```

3. NETWORK ADAPTORS

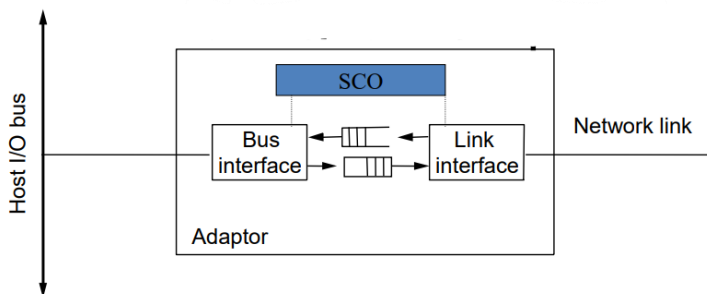
Network adaptors (scheda di rete): interfaccia il computer alla rete



La scheda è costituita da due parti separate che interagiscono attraverso una FIFO che maschera l'asincronia tra la rete e il bus interno (bus interno e rete potrebbero avere data rate diversi).

La prima parte interagisce con la CPU della scheda, la seconda parte interagisce con la rete (implementando il livello fisico e di collegamento).

Tutto il sistema è comandato da una **SCO** (sottosistema di controllo della scheda).



L'adaptor esporta verso la CPU uno o più registri macchina (Control Status Register).

CSR è il mezzo di comunicazione tra la SCO della scheda e la CPU.

```
/* CSR
 * leggenda: RO - read only; RC - Read/Clear (writing 1 clear, writing 0 has no effect);
 * W1 write-1-only (writing 1 sets, writing 0 has no effect)
 * RW - read/write; RW1 - Read-Write-1-only
 */
#define LE_ERR 0X8000
.....
#define LE_RINT 0X0400 /* RC richiesta di interruzione per ricevere un pacchetto */
#define LE_TINT 0X0200 /* RC pacchetto trasmesso */
.....
#define LE_INEA 0X0040 /* RW abilitazione all'emissione di un interrupt da parte
.....
                        dell'adaptor verso la CPU */
#define LE_TDMD 0X0008 /* W1 richiesta di trasmissione di un pacchetto dal device
                        driver verso l'adaptor */
```

L'host può controllare cosa accade in CSR in due modi:

- Busy waiting (la CPU esegue un test continuo di CSR fino a che CSR non si modifica indicando la nuova operazione da eseguire. Ragionevole solo per calcolatori che non devono fare altro che attendere e trasmettere pacchetti, ad esempio i router)
- Interrupt (l'adaptor invia un interrupt all'host, il quale fa partire un interrupt handler che va a leggere CSR per capire l'operazione da fare)

Trasferimento dati da adaptor a memoria (e viceversa):

Direct Memory Access (DMA):

- Nessun coinvolgimento della CPU nello scambio dati
- Il SO assegna un'area di memoria all'host
- Frame inviati immediatamente alla memoria di lavoro dell'host
- Pochi bytes di memoria necessari sulla scheda

Programmed I/O:

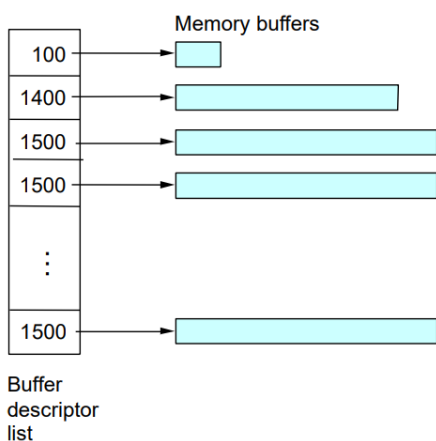
- Lo scambio dati tra memoria e adaptor passa per la CPU
- Impone di bufferizzare almeno un frame sull'adaptor
- La memoria deve essere di tipo dual port, il processore e l'adaptor possono sia leggere che scrivere in questa porta

PMA:

La memoria dove allocare i frames è organizzata attraverso una **buffer descriptor list**, 1 in scrittura 1 in lettura.

Un vettore di puntatori ad aree di memoria (buffers) dove è descritta la quantità di memoria disponibile in quell'area.

In ethernet vengono tipicamente preallocati 64 buffers da 1500 bytes.



Tecnica usata per frame che arrivano dall'adaptor: scatter read / gather write

Frame distinti sono allocati in buffer distinti, un frame può essere allocato su più buffer, se più grande del buffer (in ethernet non necessario, perché i pacchetti vengono spezzettati e siamo sempre nei 1500 byte).

Quando un messaggio viene inviato da un utente in un certo socket:

1. Il SO copia il messaggio dal buffer della memoria utente in una zona di BD

2. Tale messaggio viene processato da tutti i livelli protocollari (esempio TCP, IP, device driver) che provvedono ad inserire gli opportuni header e ad aggiornare gli opportuni puntatori presenti nel BD in modo da poter sempre ricostruire il messaggio
3. Quando il messaggio ha completato l'attraversamento del protocol stack, viene avvertita la SCO dell'adaptor dal device driver attraverso il set dei bit del CSR (LE_TDMD e LE_INEA). Il primo invita la SCO ad inviare il messaggio sulla linea. Il secondo abilita la SCO ad inviare una interruzione
4. La SCO dell'adaptor invia il messaggio sulla linea
5. Una volta terminata la trasmissione, la SCO notifica il termine alla CPU attraverso il set del bit (LE_TINT) del CSR e scatena una interruzione
6. Tale interruzione avvia un interrupt handler che prende atto della trasmissione, resetta gli opportuni bit (LE_TINT e LE_INEA) e libera le opportune risorse (operazione semsignal su xmit_queue)

Device driver: è una collezione di routine (inizializzare l'adaptor, invio di un frame sul link etc.) di OS che serve per "ancorare" il SO all'hardware sottostante specifico dell'adaptor

es. routine di richiesta di invio di un messaggio sul link

```
#define csr ((u_int) 0xffff3579 /*CSR address*/
Transmit(Msg *msg)
{
    descriptor *d;
    semwait(xmit_queue);      /* abilita non piu' di 64 accessi al BD*/ semaforo inizializzato precedentemente a 64 (ho 64 buffer)
    d=next_desc();
    prepare_desc(d,msg);
    semwait(mutex);          /* abilita a non piu' di un processo (dei potenziali 64) alla volta la trasmissione verso l'adaptor */
    disable_interrupts();     /* il processo in trasmissione si protegge da eventuali interruzioni dall'adaptor */ disabilita gli interrupt
    csr= LE_TDMD | LE_INEA;   /* una volta preparato il messaggio invita la SCO dell'adaptor a trasmetterlo e la abilita la SCO ad emettere una interruzione una volta terminata la trasmissione */
    enable_interrupts();      /* riabilita le interruzioni */
    semsignal(mutex);         /* sblocca il semaforo per abilitare un altro processo a trasmettere */
}
```

"next_desc()" ritorna il prossimo buffer descriptor disponibile nel buffer descriptor list.

"prepare_desc(d,msg)" il messaggio msg nel buffer d in un formato comprensibile dall'adaptor.

Interrupt Handler

- Disabilita le interruzioni
- Legge il CSR per capire che cosa deve fare: tre possibilità
 1. C'è stato un errore
 2. Una trasmissione è stata completata
 3. Un frame è stato ricevuto
- Noi siamo nel caso 2
 - LE_TINT viene messo a zero (RC bit)
 - Ammette un nuovo processo nella BD poiché un frame è stato trasmesso
 - Abilita le interruzioni

```

lance_interrupt_handler()
{
    disable_interrupts();

    /* some error occurred */
    if (csr & LE_ERR)
    {
        print_error(csr);
        /* clear error bits */
        csr = LE_BABL | LE_CERR | LE_MISS | LE_MERR | LE_INEA;
        enable_interrupts();
        return();
    }

    /* transmit interrupt */
    if (csr & LE_TINT)
    {
        /* clear interrupt */
        csr = LE_TINT | LE_INEA;

        /* signal blocked senders */
        semSignal(xmit_queue);

        enable_interrupts();
        return(0);
    }

    /* receive interrupt */
    if (csr & LE_RINT)
    {
        /* clear interrupt */
        csr = LE_RINT | LE_INEA;

        /* process received frame */
        lance_receive();
        enable_interrupts();
        return();
    }
}

```

Meccanismo di funzionamento di una scheda di rete:

